Data.Singleton: https://hackage.haskell.org/package/singletons
Data.Singleton.TH: https://hackage.haskell.org/package/singletons-th
Introduction to singletons: https://blog.jle.im/entries/series/+introduction-to-singletons.html

Data.Singleton

```
type Sing :: forall k. k -> Type
type family Sing :: k -> Type
pattern Sing :: forall k (a :: k). () => SingI a => Sing a


class SingI a where                                                    --
e.g. SingI 'True      sing = STrue
 sing :: Sing a                                                        --
        SingI 'False     sing = SFalse

class SingKind k where                                                 --
k as kind
 type Demote k = (r :: Type) | r -> k                                  --
e.g. SingKind Bool        Demote Bool = Bool
 fromSing :: Sing (a :: k) -> Demote k                                 --
                          fromSing STrue = True
 toSing :: Demote k -> SomeSing

data SomeSing k where
  SomeSing :: forall k (a :: k). Sing a -> SomeSing k                  --
k as kind

withSing :: SingI a => (Sing a -> b) -> b
withSingI :: Sing n -> (forall a. SingI a => r) -> r
withSomeSing :: forall r k. SingKind k
  => Demote k -> (forall (a :: k). Sing a -> r) -> r



type family Sing :: k -> Type
data SomeSing k where
  SomeSing :: Sing (a :: k) -> SomeSing k                              --
Existential


-- Singleton
class SingI a where
  sing :: Sing a

withSingI :: Sing n -> (SingI n => r) -> r
withSing :: SingI a => (Sing a -> b) -> b
withSomeSing :: forall k r. SingKind k => Demote k -> (forall (a ::
k). Sing a -> r) -> r
singByProxy :: SingI a => proxy a -> Sing a
```

```
-- Higher-kinded Singleton
class (forall x. SingI x => SingI (f x)) => SingI1 f where
  liftSing :: Sing x -> Sing (f x)
class (forall x y. (SingI x, SingI y) => SingI (f x y)) => SingI2 f
where
  liftSing2 :: Sing x -> Sing y -> Sing (f x y)

sing1 :: (SingI1 f, SingI x) => Sing (f x)
sing2 :: (SingI2 f, SingI x, SingI y) => Sing (f x y)
withSing1 :: (SingI1 f, SingI x) => (Sing (f x) -> b) -> b
withSing2 :: (SingI2 f, SingI x, SingI y) => (Sing (f x y) -> b) ->
b
usingSingI1 :: forall f x r. (SingI1 f, SingI x) => (SingI (f x) =>
r) -> r
usingSingI2 :: forall f x y r. (SingI2 f, SingI x, SingI y) =>
(SingI (f x y) => r) -> r
singByProxy1 :: (SingI1 f, SingI x) => proxy (f x) -> Sing (f x)
singByProxy2 :: (SingI2 f, SingI x, SingI y) => proxy (f x y) ->
Sing (f x y)singByProxy# :: SingI a => Proxy# a -> Sing a
singByProxy1# :: (SingI1 f, SingI x) => Proxy# (f x) -> Sing (f x)
singByProxy2# :: (SingI2 f, SingI x, SingI y) => Proxy# (f x y) ->
Sing (f x y)


-- Instance
data SingInstance (a :: k) where
  SingInstance :: SingI a => SingInstance a
singInstance :: forall k (a :: k). Sing a -> SingInstance a



-- Convert type <=> term e.g. 'True :: Bool <=> True <=> Bool
class SingKind k where
  type Demote k = (r :: Type) | r -> k
  fromSing :: Sing (a :: k) -> Demote k
        -- type to term
  toSing :: Demote k -> SomeSing k
        -- term to type

type KindOf (a :: k) = k
type SameKind (a :: k) (b :: k) = () :: Constraint
pattern FromSing :: SingKind k => forall (a :: k). Sing a -> Demote
k

demote :: forall a. (SingKind (KindOf a), SingI a) => Demote
(KindOf a)
demote1 :: forall f x. (SingKind (KindOf (f x)), SingI1 f, SingI x)
=> Demote (KindOf (f x))
demote2 :: forall f x y. (SingKind (KindOf (f x y)), SingI2 f,
SingI x, SingI y) => Demote (KindOf (f x y))
singThat :: forall k (a :: k). (SingKind k, SingI a) => (Demote k
-> Bool) -> Maybe (Sing a)
singThat1 :: forall k1 k2 (f :: k1 -> k2) (x :: k1). (SingKind k2,
```

```
SingI1 f, SingI x) => (Demote k2 -> Bool) -> Maybe (Sing (f x))
singThat2 :: forall k1 k2 k3 (f :: k1 -> k2 -> k3) (x :: k1) (y ::
k2). (SingKind k3, SingI2 f, SingI x, SingI y) => (Demote k3 ->
Bool) -> Maybe (Sing (f x y))
```

Basics

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE InstanceSigs #-}
{-# LANGUAGE KindSignatures #-}

import Data.Singletons
import Data.Singletons.TH

genSingletons [''Bool]


-- Singleton
--
-- instance
STrue :: Sing 'True                    -- singleton
sing @'True :: Sing 'True              -- same with sing function
Sing  :: Sing 'True                    -- same with Sing pattern

-- pattern match
{-# LANGUAGE LambdaCase #-}
f :: SingI (a :: Bool) => Proxy a -> String
f _ = withSing @a $ \case
 STrue -> ""
 SFalse -> ""
f _ = case sing @a of                  -- same
 STrue -> ""
 SFalse -> ""


-- Demote i.e. convert type <-> term
--
-- type -> term
fromSing STrue :: Bool                 -- True
demote @'True :: Bool                  -- same

-- term -> type
toSing True :: SomeSing Bool           -- SomeSing STrue
```

==========

Singletons with user-defined types

Example: Bool (nullary), Maybe (nary) singletons
e.g. singleton

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE KindSignatures #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE RankNTypes #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TemplateHaskell #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE UndecidableInstances #-}
{-# LANGUAGE LambdaCase #-}
{-# LANGUAGE EmptyCase #-}
{-# OPTIONS_GHC -Werror=incomplete-patterns #-}

import Data.Singletons
import Data.Singletons.TH

genSingletons [''Bool, ''Maybe]
```

Equivalent Bool

```
{-# LANGUAGE InstanceSigs #-}

data SBool s where
  STrue :: SBool 'True
  SFalse :: SBool 'False

type instance Sing = SBool
instance SingI 'True where
  sing :: SBool 'True
  sing = STrue
instance SingI 'False where
  sing :: SBool 'False
  sing = SFalse

instance SingKind Bool where                    -- SingKind (Bool
as kind) i.e. 'True :: Bool 'False :: Bool
  type Demote Bool = Bool                       -- Demote (Bool as
kind) = (Bool as Type)

  fromSing :: SBool s -> Bool
  fromSing STrue = True
  fromSing SFalse = False
```

```
  toSing :: Bool -> SomeSing Bool
  toSing True = SomeSing STrue
  toSing False = SomeSing SFalse

instance SDecide Bool where
  STrue %~ STrue = Proved Refl
  SFalse %~ SFalse = Proved Refl
  STrue %~ SFalse = Disproved $ \case
  SFalse %~ STrue = Disproved $ \case
```

e.g.

```
STrue :: SBool 'True            -- singleton of 'True
sing @'True :: Sing 'True       -- same

SFalse :: SBool 'False          -- singleton of 'False
sing @'False :: Sing 'False

Demote Bool                      -- Bool as type
True :: Demote Bool
False :: Demote Bool
```

Equivalent Maybe

```
type SMaybe :: forall k. Maybe k -> Type
data SMaybe k where
  SNothing :: SMaybe 'Nothing
  SJust :: Sing x -> SMaybe ('Just x)

type instance Sing = SMaybe

instance SingI x => SingI ('Just x) where
  sing :: SMaybe ('Just x)
  sing = SJust (sing @x)

instance SingI ('Nothing) where
  sing :: SMaybe ('Nothing)
  sing = SNothing

instance SingKind k => SingKind (Maybe k) where
  type Demote (Maybe k) = Maybe (Demote k)

  fromSing :: Sing (l :: Maybe k) -> Demote (Maybe k)
  fromSing (SJust x) = Just $ fromSing x
  fromSing SNothing = Nothing

  toSing :: Demote (Maybe k) -> SomeSing (Maybe k)
  toSing Nothing = SomeSing SNothing
  toSing (Just x) = withSomeSing x (SomeSing . SJust)

instance SDecide a => SDecide (Maybe a) where
```

```
  SNothing %~ SNothing = Proved Refl
  SNothing %~ SJust _ = Disproved $ \case
  SJust _ %~ SNothing = Disproved $ \case
  SJust v  %~ SJust w = case v %~ w of
                          Proved Refl  -> Proved Refl
                          Disproved rv -> Disproved $ \case Refl ->
rv Refl
```

e.g.

```
SJust STrue :: SMaybe ('Just 'True)           -- singleton of
('Just 'True)
sing @('Just 'True) :: Sing ('Just 'True)     -- same

SNothing :: SMaybe ('Nothing)                 -- singleton
of 'Nothing
sing @'Nothing :; Sing 'Nothing               -- same
```

Usage: define singletons manually

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE InstanceSigs #-}
{-# LANGUAGE KindSignatures #-}

import Data.Kinds
import Data.Singletons

-- Step: define kind
type MyValue :: Type
data MyValue = A | B                          -- type=A kind=MyValue


-- Step: define singletons
type SMyValue :: MyValue -> Type
data SMyValue a where
 SA :: SMyValue 'A                            -- singleton for 'A
 SB :: SMyValue 'B


-- Step: register
type instance Sing = SMyValue                 -- IMPORTANT: this only
works because SMyValue is kind restricted SMyValue :: MyValue ->
Type
                                              -- i.e. fails if
SMyValue :: forall a. a -> Type
instance SingI 'A where
```

```
  sing :: SMyValue 'A
  sing = SA
instance SingI 'B where
  sing :: SMyValue 'B
  sing = SB
```

==========

Type-level functions

Prelude.Singletons: https://hackage.haskell.org/package/singletons-base-3.1.1/docs/Prelude-Singletons.html

Type-level functions
Compute a complex type

x :: Sing 'True
x :: Sing (1 == 1) same computed

e.g. typing a value

```
STrue :: Sing 'True
STrue :: Sing (1 == 1)                              -- same
STrue :: Sing (Fst '( 'True, 'False)                -- same with
Fst
STrue :: Sing (Maybe_ 'False IdSym0 ('Just 'True)) -- same with
Maybe_
STrue :: Sing ('[1,2] == '[1] <> '[2])              -- same with
monoid <>
```

Basics

```
import Prelude.Singletons

-- e.g. conditional operators
type X :: Bool
type X = 1 == 1                                     -- X = 'True
type X = 1 \= 1                                     -- X = 'False
type X = 'True == 'True
type X = "A" == "A"
type X = 1 == 'True                                 -- ERROR
type A = 1
type B = 1
type X = A == B                                     -- note: NOT the
same as A ~ B.
                                                    -- A ~ B ::
Constraint and A == B :: Bool
type X = 1 < 2
type X = 1 > 2
type X = 1 <= 2
type X = 1 == 1 && 2 == 2
type X = 1 == 1 || 2 == 2
```

```
-- e.g. arithmetic
type X :: Nat
type X = 1 + 2            -- X=3
type X = 1 - 2            -- X=Negate 1
type X = 1 - Negate 2     -- X=3
type X = Negate 1
type X = Negate (Negate 1)    -- X=1
type X = Abs 1


-- e.g. functions
type X = Maybe_ 0 IdSym0 ('Just 1) :: Nat                        --
X=1     i.e. same as (maybe 0 id (Just 1))
type X = Maybe_ 0 IdSym0 Nothing :: Nat                          --
X=0
type X = Either_ ((+@#@$$) 1) ((+@#@$$) 2) ('Left 1) :: Nat      --
X=2     i.e. same as (either (+1) (+2) (Right 1))
type X = Either_ ((+@#@$$) 1) ((+@#@$$) 2) ('Right 1)           --
X=3


-- e.g. tuple
type X = '(1,2) :: (Nat, Nat)
type X = Fst '(1,2) :: Nat            -- X=1
type X = Snd '(1,2) :: Nat            -- X=2


-- e.g. semigroup, monoid
type X = 1 <> 2 :: Nat                    -- X=3
type X = "A" <> "B" :: Symbol             -- X="AB"
type X = '[3] <> '[1, 2] :: [Nat]         -- X='[3,1,2]
type X = Mempty :: [Nat]                  -- X='[]
type X = MAppend 1 2 :: Nat               -- X=3
type X = Mconcat '[1,2,3] :: Nat          -- X=6


-- e.g.
type X = Fmap ((*@#@$$) 3) '[ 1,2,3]           -- X='[3,6,9]
i.e. fmap (*3)

-- e.g. list
type X :: [Nat]
type X = '[]
type X = '[ 1,2,3]
type X = Filter ((<=@#@$$) 2) '[ 1,2,3]        -- X = '[1,2]
  i.e. filter (<2)
type X = Map ((+@#@$$) 2) '[ 1,2,3]            -- X = '[ 3,4,5]
  i.e. map (+2)
```

Defunctionalization
Ref: https://hackage.haskell.org/package/singletons-3.0.2/docs/Data-Singletons.html

```
-- Data.Singletons
data TyFun :: Type -> Type -> Type
type x ~> y = TyFun x y -> Type                          -- e.g.
type F :: x ~> y
                                                         --
data F (a :: TyFun x y)

type Apply :: forall a b. (a ~> b) -> a -> b
type family Apply (f :: a ~> b) (a :: Type) :: b
type a @@ b = Apply a b
```

Defunctionalization
Representing **partially applied type functions** for use in **higher-order type functions**
e.g. Foldr
Normal type functions (i.e. type family) can NOT be partially applied (due to **arity**: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/type_families.html#type-family-declarations)
A **defunctionalized** function is a **dummy type** representing a partially applied function

type Max :: a -> b -> c	original type-level function Max x y	
type MaxSym1 x0 :: b ~> c	defunctionalized (Max x0)	i.e. partially applied b
type MaxSym0 :: a ~> b ~> c	defunctionalized Max	fully unapplied a b
Max 1 2	usual application	
MaxSym1 1 @@ 2	@@ = apply defunctionalized parameter	
MaxSym0 @@ 1 @@ 2		

e.g. usage in higher order function

```
type Max :: a -> a -> a                    -- original function
                                 type family Max x y
type MaxSym0 :: a ~> (a ~> a)              -- Defunctionalized Max
i.e. both vars unapplied        data MaxSym0 a
type MaxSym1 :: a -> (a ~> a)              -- Max x0 y i.e. one
variable applied                   data MaxSym1 x0 a

-- e.g. Function call
type A = Max                          -- ERROR "The associated type
family 'Max' should have 2 arguments
type A = MaxSym0                      -- OK
type A = Max 10                       -- ERROR  "The associated type
family 'Max' should have 2 arguments
type A = MaxSym1 10                   -- OK. Represents Max 10 y
type A = MaxSym0 @@ 10                -- same
type A = Max 10 20                    -- 20
type A = MaxSym1 10 @@ 20             -- same
type A = MaxSym0 @@ 10 @@ 20          -- same
```

```
-- e.g. Higher order function call
type Foldr :: (a ~> (b ~> b)) -> a -> t a -> t a
type A = Foldr MaxSym0 1 '[ 0, 1, 2]        -- OK  A = 2
type A = Foldr Max 1 '[ 0, 1, 2]            -- ERROR "The
associated type family 'Max' should have 2 arguments"

type Map :: (a ~> b) -> t a -> t b
type A = Map (MaxSym1 1) '[ 0, 1, 2]        -- OK A = '[1, 1, 2]
type A = Map (MaxSym0 @@ 1) '[ 0, 1, 2]     -- same
type A = Map (Max 1) '[ 0, 1, 2]            -- ERROR "The
associated type family 'Max' should have 2 arguments"
```

usage: defining defunctionalized symbols
e.g.

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE UndecidableInstances #-}

import Data.Kind
import Data.Singletons.TH
import GHC.TypeLits (Nat)
import Prelude.Singletons



-- step: define function
type F :: Nat -> Nat -> Nat                         -- original
function F
type family F a b where
  F x y = x + y

-- step : define defunctionalized
type FSym0 :: (Nat ~> (Nat ~> Nat))                  -- F x y with x
y unapplied
data FSym0 a                                         -- a :: TyFun
Nat (Nat ~> Nat)
type FSym1 :: Nat -> (Nat ~> Nat)                    -- F x0 y with
y unapplied
data FSym1 x a                                       -- x :: Nat,
a :: TyFun Nat Nat

-- step: define implementation
type instance Apply FSym0 x = FSym1 x
type instance Apply (FSym1 x) y = F x y
```

```
-- e.g. F application
type A :: Nat
type A = F 1 2                                  -- F 1 2 = 3
type A = Apply (FSym1 1) 2                      -- same
type A = FSym1 1 @@ 2
type A = Apply (Apply (FSym2 1)) 2
type A = FSym1 @@ 1 @@ 2



-- e.g. higher order
type A :: Nat
type A = Foldr FSym0 0 '[1, 2, 3]                    -- A = 6
type A :: Nat
type A = FoldrSym0 @@ FSym0 @@ 0 @@ '[1, 2, 3]      -- same

type A :: [Nat]
type A = Map (FSym1 10) '[1, 2, 3]                   -- A = '[ 11,
12, 13]
```

Defunctionalization basics

```
type F :: a ~> b
type F :: TyFun a b -> Type                          -- same
data F (f :: TyFun a b)

type F :: a ~> (b ~> c)
type F :: TyFun a (b ~> c) -> Type                   -- same
type F :: TyFun a (TyFun b c -> Type) -> Type        -- same
data F (f :: TyFun a (b ~> c)



type And :: Bool -> Bool -> Bool                     -- original
function And x y (type family)
type AndSym1 :: Bool -> (Bool ~> Bool)               -- one
variable applied i.e. A x0 y
type AndSym0 :: Bool ~> (Bool ~> Bool)               -- all
unapplied
```

```
type A = And
type A = AndSym0                    :: Bool ~> (Bool ~> Bool)
type A = And 'True
type A = AndSym1 'True              :: Bool ~> Bool
type A = AndSym0 @@  'True          :: Bool ~> Bool
type A = And 'True 'False           :: Bool
type A = AndSym1 'True @@ 'False    :: Bool
type A = AndSym0 @@ 'True @@ 'False  :: Bool
```

==========

OLD

OLD

Usage: convert type <=> term i.e. n <=> Sing n

sing :: SingI n => Sing n type to singleton i.e. n to Sing n
withSing :: Sing n -> (SingI n => r) -> r singleton to type i.e. Sing n to n

e.g.

```
import Data.Singletons.TH

genSingletons [''Bool]

type MyData :: Bool -> Type
data MyData a where
  MkMyData :: SingI a => MyData a

-- case: embed a into MyData
mkMyData :: forall (a :: Bool). Sing a -> MyData a
mkMyData a = withSing a MkMyData



-- case: extract a from MyData
fromMyData :: forall (a :: Bool). MyData a -> Sing a
fromMyData MkMyData = sing @a
```

usage: reify/reflect with SingKind
Each way requires 2 steps

reify value :: a -> Sing a -> SingI a -> type a
  toSing (value -> Sing a) + withSing (Sing a -> SingI a)
  withSomeSing (value -> Sing a) + withSing (Sing a -> SingI a)

reflect type a -> SingI a -> Sing a -> value
  sing (SingI a -> Sing a) + fromSing (Sing a -> value :: a)

```
-- case: reify with toSing
toSing :: SingKind k => Demote k -> SomeSing
withSingI :: Sing a -> (SingI a => r) -> r
reifyValue :: forall k. SingKind k => Demote k -> ...
reifyValue v = use . toSing                                  --
toSing : value -> Sing a
  where
   use (SomeSing (s :: Sing a)) = withSing s $               --
withSingI: Sing a -> SingI a
         ... use SingI a ...
```

```
-- case: reflect with fromSing
sing :: SingI a => Sing a
fromSing :: SingKind k => Sing (a :: k) -> Demote k
reflectType :: forall k (a :: k) p. (SingKind k, SingI a) => p a ->
Demote k
reflectType _ = fromSing (sing @a)                          --
sing: SingI a -> Sing a
                                                            --
fromSing :: Sing a -> value
```

e.g.

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TemplateHaskell #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE UndecidableInstances #-}
{-# LANGUAGE ConstraintKinds #-}

import Data.Kind
import Data.Singletons
import Data.Singletons.TH

genSingletons [''Bool, ''Maybe]

-- setup: Proxy container
type ProxyKind k = (SingKind k, Show (Demote k))
type SomeProxy :: Type
data SomeProxy where
  MkSomeProxy :: forall k (a :: k). (ProxyKind k, SingI a) => Proxy
a -> SomeProxy

instance Show SomeProxy where
  show (MkSomeProxy p) = "SomeProxy: " ++ show (reflectProxy p)

-- case:
mkSomeProxy :: forall k (a :: k). (ProxyKind k) => Sing a ->
SomeProxy
mkSomeProxy s = withSingI s $ MkSomeProxy (Proxy @a)

-- case: reify
-- i.e. value -> opaque contianer
reifyProxy :: forall k. ProxyKind k => Demote k -> SomeProxy
reifyProxy = use . toSing
    where
      use (SomeSing (s :: Sing a)) = withSingI s $ MkSomeProxy
(Proxy @a)
```

```
-- case: reflect
-- i.e. value
reflectProxy :: forall k (a :: k). (ProxyKind k, SingI a) => Proxy
a -> Demote k
reflectProxy _ = fromSing (sing @a)



-- e.g.
main :: IO ()
main = do
  print $ reifyProxy True                              -- MkSomeProxy
(Proxy @'True)
  print $ reifyProxy False                             -- MkSomeProxy
(Proxy @'False)
  print $ reifyProxy $ Just True                       -- MkSomeProxy
(Proxy @('Just 'False)
  print $ reifyProxy @(Maybe Bool) $ Nothing           -- MkSomeProxy
(Proxy @Nothing)

  print $ mkSomeProxy STrue
  print $ mkSomeProxy SFalse
  print $ mkSomeProxy $ SJust SFalse
  print $ mkSomeProxy @(Maybe Bool) SNothing
```